

Chapter 03 — M3: Dependency Mapping

Feature type: Mandatory analysis module (M3). **Source:** `backend/src/ai/nodes/m3Dependency.node.ts`, `backend/src/ai/prompts/m3Prompt.ts` **Output type:** `IM3Result = { graph: IDependencyNode[], formattedAscii: string }`

1. What it does

M3 builds the **module dependency graph** — how files import each other — and returns:

- `graph` — per file: `{ file, imports[], importedBy[] }` (forward **and** reverse edges)
- `formattedAscii` — a human-readable ASCII tree of the dependency structure

This is the "how do the pieces connect" layer, and it powers the interactive graph views in the UI.

2. How it works (step by step)

```
parsedRepo.files                                each: { filePath, imports[], exports[], functions[], ... }
  |
  ▼
guard: no files → { graph:[], formattedAscii:"" } + "[M3] No files to map"
  |
  ▼
cap at MAX_FILES_FOR_LLM = 40    files.slice(0, 40)    ← token safeguard
  |
  ▼
keep LOCAL imports only          imports.filter(i => i.startsWith(".") || i.startsWith("/"))
                                  (drops npm/stdlib packages – internal edges only)
  |
  ▼
LLM call (M3Model.withStructuredOutput(M3OutputSchema))
  system: M3_SYSTEM_PROMPT, user: buildM3UserPrompt(dependencyData)
  |
  ▼
m3Result : { graph, formattedAscii }
```

Two safeguards worth knowing:

1. **40-file cap** (`MAX_FILES_FOR_LLM`) — dependency chains explode combinatorially; capping protects the token budget and latency.
 2. **Local-imports-only filter** — only relative (`.` / `/`) imports are kept, so the graph shows the project's *own* structure, not `react` / `express` /etc.
-

3. The model behind it

- **Model:** `M3Model = NVIDIA nemotron-3-super-120b-a12b`, `temperature: 0`.
 - **Why the big model:** M3 must emit well-formed structured JSON (nodes + reverse edges + an ASCII tree) — the larger Nemotron is the most reliable at this structured-output task.
-

4. Output shape

```
{
  "graph": [
    { "file": "src/server.ts", "imports": ["src/app.ts"], "importedBy": [] },
    { "file": "src/app.ts", "imports": ["src/routes/x.ts"], "importedBy": ["src/server.ts"] }
  ],
  "formattedAscii": "src/server.ts\n└─ src/app.ts\n    └─ src/routes/x.ts"
}
```

Schema note: the Mongoose model stores `result.m3` as an **array** of `DependencyNodeSchema` (`m3: [DependencyNodeSchema]`), while the runtime type `IDependencyMap / IM3Result` is an **object** `{ graph, formattedAscii }`. This mismatch means `formattedAscii` is not persisted by the current schema — flagged in `plan.md`.

5. Failure behavior

- No files → empty graph + `[M3] No files to map`.
 - Thrown error → `{ graph:[], formattedAscii:"" }` + `[M3] <message>`.
 - Missing at Combine → `m3` becomes `{ graph:[], formattedAscii:"" }` + warning.
-

6. Where it surfaces in the UI

`data.m3.graph[]` → `graphNodes` + `graphEdges`, rendered with **ReactFlow** + **dagre** in both **DashboardPage** and **DependencyGraphPage**. Node click → side panel with imports / imported-by / AI summary / complexity / risk.

7. Known gaps / improvement ideas

- 40-file cap silently truncates large repos — the graph may miss modules.
- `importedBy` correctness depends on the LLM; there's no post-hoc verification that reverse edges match forward edges.

- Import path resolution is textual (no alias/tsconfig-paths resolution), so `@/x` style imports won't match a real file.
- Persist `formattedAscii` (fix the schema array-vs-object mismatch).